LyASC: LYAPUNOV-AWARE SKIP CONNECTIONS FOR STABILITY-DRIVEN RESIDUAL NETWORKS

A PROJECT REPORT

Submitted by

SUBHASH R

(Reg. No. CH.SC.U4AIE23054)

VEENA SUNDARI A

(Reg. No. CH.SC.U4AIE23060)

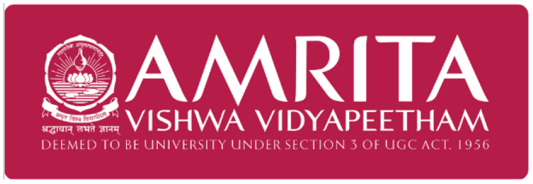
in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)

Under the guidance of

Dr. RITHANI M

Submitted to



AMRITA VISHWA VIDYAPEETHAM

AMRITA SCHOOL OF COMPUTING

CHENNAI – 601103

October 2026



BONAFIDE CERTIFICATE

This is to certify that this project report entitled **"LyASC: Lyapunov-Aware Skip Connections for Stability-Driven Residual Networks"** is the bonafide work of **"Subhash R (CH.SC.U4AIE23054) and Veena Sundari A (CH.SC.U4AIE23060)"** who carried out the project work under my supervision.

SIGNATURE

Dr. S. BAGHAVATHI PRIYA
CHAIRPERSON
Department of CSE
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Chennai - 601103

SIGNATURE

Dr. Rithani M
SUPERVISOR
Department of CSE
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Chennai - 601103

INTERNAL EXAMINER

EXTERNAL EXAMINER



DECLARATION BY THE CANDIDATE

We hereby declare that the project report entitled **"LyASC: Lyapunov-Aware Skip Connections for Stability-Driven Residual Networks"** is a bonafide record of the Phase 2 work done by us under the guidance of our supervisor. This work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or other similar title to any candidate of any University.

Subhash R
(CH.SC.U4AIE23054)

Veena Sundari A
(CH.SC.U4AIE23060)

ABSTRACT

Deep residual networks rely on skip connections to carry information through the network, but the weight of the skip path is usually fixed or set by learned parameters, with no signal that describes how the residual branch is behaving locally. This project implements and evaluates LyASC, a stability-informed skip-weighting method, inside PreAct-ResNet-110 on CIFAR-10. The implementation uses a finite, smoothed Jacobian-vector amplification proxy computed from a probe perturbation. It does not estimate the dominant singular value directly, and it does not measure the stability of the whole network. Under standard training, the three-seed LyASC+LN configuration reached a mean test error of 7.17%. That is lower than the reproduced wSkip+LN mean of 8.16% and higher than the 1xSkip and 2rSkip+LN means of 6.27% and 6.15%. In the stress_0.5 and stress_2.0 settings, all three LyASC runs had test errors below 9%. The five stress_3.0 seeds varied widely. The cold-start and post-gate ablations also gave different observed means, and the seed counts differed between configurations. The results are descriptive and apply only to the evaluated architecture, settings, and recorded runs. They do not show a general performance advantage or a formal stability guarantee.

Keywords: LyASC, residual networks, skip connections, PreAct-ResNet, Jacobian-vector product, CIFAR-10, stability proxy

ACKNOWLEDGEMENT

This project work would not have been possible without the contribution of many people. It gives me immense pleasure to express my profound gratitude to our Honorable Chancellor, **Sri Mata Amritanandamayi Devi**, for her blessings and inspiration. I am deeply indebted to our Administrative Director, **Swami Vinayamritananda Puri**, and Campus Director, **Mr. I B Manikantan**, Amrita School of Computing and Engineering, for providing the facilities and support that made this learning experience possible.

I thank **Dr. V. Jayakumar**, Principal, Amrita School of Computing and Engineering, for the support extended to me during this project. I sincerely thank **Dr. S. Baghavathi Priya**, Chairperson, **Dr. R. Annamalai**, Program Chair, and my supervisor, **Dr. Rithani M**, Department of Computer Science and Engineering, for their guidance, personal involvement, and constant encouragement throughout this work.

I am grateful to the Project Coordinator, the Review Panel Members, and the faculty of the Department of Computer Science and Engineering for their criticism and suggestions, which helped improve the quality of this work.

Subhash R

(CH.SC.U4AIE23054)

Veena Sundari A

(CH.SC.U4AIE23060)

TABLE OF CONTENTS

Abstract iv

Acknowledgement v

List of Tables ix

List of Figures x

List of Symbols and Abbreviations xi

1 INTRODUCTION 1

1.1 Background: Deep Learning and Residual Networks 1
1.2 Motivation 1
1.3 Problem Statement 2
1.4 Research Objectives 2
1.5 Scope and Contributions 3
1.6 Report Organization 3

2 LITERATURE REVIEW 4

2.1 Residual Learning and Identity Mappings 4
2.2 Skip Scaling and Normalization 4
2.3 Learned Gates and Dynamic Routing 5
2.4 Optimization Stability and Lyapunov Analysis 5
2.5 Comparative Analysis 5
2.6 Eighteen-Reference Survey Map 6
2.7 Research Gap 6

3 BASE PAPER REPRODUCTION 7

3.1 Purpose of Reproduction 7
3.2 Reference Architecture and Dataset 7
3.3 Reproduction Protocol 7
3.4 Baseline Results 7
3.5 What the Reproduction Contributes to LyASC 8
3.6 Reproduction Limitations 8

4 LyASC METHOD AND MATHEMATICAL FORMULATION 9
4.1 Overview . . . 9
4.2 Residual-Block Formulation . . . 9
4.3 Stability Proxy . . . 9
4.4 Exponential Smoothing . . . 10
4.5 Adaptive Gate . . . 10
4.6 Controller Interpretation . . . 11
4.7 Integration with PreAct-ResNet-110 . . . 11
4.8 Method Summary . . . 12

5 SYSTEM DESIGN AND IMPLEMENTATION 13
5.1 System Architecture . . . 13
5.2 Software Environment . . . 13
5.3 Stability Estimator Module . . . 13
5.4 Gate and Model Integration . . . 13
5.5 Training and Evaluation Pipeline . . . 14
5.6 Testing and Verification . . . 14
5.7 Experiment Tracking and Dashboard . . . 15
5.8 Reproducibility Safeguards . . . 15

6 EXPERIMENTAL DESIGN AND RESULTS 16
6.1 Experimental Questions . . . 16
6.2 Experimental Protocol . . . 16
6.3 Baseline Reproduction Results . . . 16
6.4 Phase C: Un-normalized LyASC . . . 17
6.5 Phase D: Cold-Start Strategy Ablation . . . 17
6.6 Phase E1: Stress Testing . . . 18
6.7 Stress_3.0: Outcome Clusters . . . 19
6.8 Phase E2: Post-Gate Normalization . . . 21
6.9 Runtime and Tracking . . . 22
6.10 Results Summary . . . 24

7 DISCUSSION, LIMITATIONS, AND VALIDITY 25
7.1 Interpretation of Main Findings . . . 25
7.2 Normal-Condition Performance . . . 25
7.3 Interpretation of Stress_3.0 . . . 25
7.4 Estimator Validity . . . 25
7.5 Statistical Limitations . . . 26

7.6 Computational Cost 26
7.7 Reproducibility and Internal Validity 26
7.8 External Validity 27
7.9 Threats to Validity 27

8 CONCLUSION AND FUTURE WORK 28
8.1 Conclusion 28
8.2 Key Contributions 28
8.3 Future Work 29

A APPENDIX - SUPPLEMENTARY MATERIAL 32
A.1 Key Configuration Values 32
A.2 Per-Seed Stress Results 32
A.3 Implementation Verification 32
A.4 Evidence Artifact Selection 33
A.5 Reference Verification Note 33

LIST OF TABLES

2.1 Research families and their relationship to LyASC 6

6.1 Phase D cold-start strategies 17
6.2 Stress-test comparison 18
6.3 Post-gate normalization pairing 22

A.1 Individual-seed stress results 32

LIST OF FIGURES

3.1 Reproduced base-paper results across selected Table 1 variants. 8

4.1 LyASC block architecture showing identity, residual, and stability-estimation paths. 11
4.2 Project workflow from baseline reproduction through stress testing and ablation. . 12

5.1 Terminal evidence for the automated test suite. 14
5.2 MLflow experiment runs table used to track experimental executions. 15

6.1 Phase C training curves for the un-normalized LyASC configuration. 17
6.2 Cold-start strategy comparison. 18
6.3 Stress-test error comparison between fixed baselines and LyASC. 19
6.4 Stress_3.0 outcomes grouped into full- and partial-rescue clusters. 20
6.5 Training-loss trajectories for stress_3.0 runs. 20
6.6 Mean gate trajectories under different stress levels. 21
6.7 Comparison of post-gate normalization variants. 22
6.8 Timing and overhead comparison across model configurations. 23
6.9 Experiment timeline recorded across project phases. 23

LIST OF SYMBOLS AND ABBREVIATIONS

AI	–	Artificial Intelligence
BN	–	Batch Normalization
CIFAR	–	Canadian Institute for Advanced Research image dataset
EMA	–	Exponential Moving Average
FTLE	–	Finite-Time Lyapunov Exponent
JVP	–	Jacobian-Vector Product
LN	–	Layer Normalization
LyASC	–	Lyapunov-Adaptive Skip Connections
PreAct	–	Pre-Activation
ResNet	–	Residual Network
SD	–	Standard Deviation
SGD	–	Stochastic Gradient Descent

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND: DEEP LEARNING AND RESIDUAL NETWORKS

Deep neural networks learn hierarchical representations by composing parameterized transformations. Greater depth lets a network express more complex functions, but it also makes optimization more sensitive to how activations and gradients propagate through many layers. Residual networks ease part of this difficulty. A block keeps its input available through a shortcut and learns only a residual transformation on top of it. The architecture became a standard foundation for image recognition and for many later neural models.

A conventional residual block can be written as

$$y_l = x_l + F_l(x_l; \theta_l),$$

where x_l is the block input, F_l is the learned transformation, and θ_l denotes the block parameters. The identity term gives the signal a direct route through the network. Pre-activation residual networks reorder the normalization and activation operations to improve signal propagation, and the PreAct-ResNet-110 backbone used in this project follows that design.

The identity pathway is useful, but its contribution is normally fixed. Other approaches use a hand-chosen scale, a learned gate, stochastic depth, or input-dependent routing. The balance between the identity and residual branches affects gradient profiles and final accuracy, so the choice of skip rule changes how the model learns.

1.2 MOTIVATION

LyASC grew out of reproducing the experiments in "Rethinking Skip Connection with Layer Normalization in Transformers and ResNets." That reproduction examined several skip scales and normalization pairings on CIFAR-10 and gave us a controlled baseline for watching how residual scaling affects optimization. We then asked whether local information about a block's amplification behaviour could adjust the identity/residual balance during training.

A fixed coefficient applies one predetermined choice to every block at every stage of training. A learned scalar is more flexible, but nothing in it is guided by a measured stability-related signal. LyASC tries a third option: estimate a local amplification proxy, smooth it over time, and use it to compute a per-block blending gate. This is a research hypothesis. We do not claim that adaptive gates must beat every fixed configuration.

1.3 PROBLEM STATEMENT

The project addresses the following question: *Can a block-local, online estimate of residual-branch amplification adapt the mixture of identity and residual pathways, and does it give measurable robustness under scaling conditions that destabilize fixed baselines?*

Answering it involves four tasks. The baseline implementation has to be faithful enough to reproduce the reference configurations. The stability proxy has to be cheap enough to compute during training. The gate has to fit into the residual block without altering unrelated training choices. And the results have to be evaluated with multiple seeds, ablations, and stated limitations.

1.4 RESEARCH OBJECTIVES

The objectives are:

1. Reproduce the selected skip-scaling and normalization baselines on CIFAR-10 using PreAct-ResNet-110.

2. Implement a block-level stability estimator based on Jacobian-vector amplification and exponential smoothing.

3. Combine the estimator with a learnable sigmoid gate that mixes the identity and residual branches.

4. Evaluate normal-condition performance and stress conditions, including contracted skip strength and expanded residual scaling.

5. Measure the effect of cold-start strategies and post-gate normalization.

6. Record accuracy, gate trajectories, gradient behaviour, timing, and seed variability.

7. Document the reproducibility safeguards and keep empirical findings separate from hypotheses.

1.5 SCOPE AND CONTRIBUTIONS

The reported experiments cover only CIFAR-10 and PreAct-ResNet-110. They say nothing about CIFAR-100, ImageNet, Transformers, or other domains. The first contribution is an implemented stability-informed gating mechanism, evaluated in stages under normal and stress settings. The second is an organized reproduction pipeline with its evidence artifacts: figures, tables, terminal captures, and run tracking.

1.6 REPORT ORGANIZATION

Chapter 1 introduces the problem and objectives. Chapter 2 reviews related research. Chapter 3 describes the reproduced base paper and the baseline evidence. Chapter 4 formalizes the LyASC method. Chapter 5 covers the implementation and experimental controls. Chapter 6 reports results and ablations. Chapter 7 discusses limitations and validity. Chapter 8 concludes and outlines future work. The appendix holds supplementary configuration, implementation notes, and result tables.

CHAPTER 2

LITERATURE REVIEW

2.1 RESIDUAL LEARNING AND IDENTITY MAPPINGS

He et al. introduced residual learning for image recognition and showed that identity shortcuts help optimize very deep networks [1]. Their formulation lets a stack of layers learn a residual function relative to its input instead of a complete transformation. A follow-up paper examined clean identity pathways and the pre-activation design [2]. Together these papers form the architectural foundation for LyASC. Neither proposes an online stability-driven gate.

Work on norm preservation studies how residual connections preserve signal or gradient norms. It helps explain why residual structures stay trainable at great depth, and it also shows that norm behaviour deserves explicit analysis. This project takes that view as its reason to measure block-level amplification instead of assuming every residual block behaves the same way.

2.2 SKIP SCALING AND NORMALIZATION

The base paper by Liu and colleagues studies skip scaling and normalization in Transformers and ResNets [3]. Its residual combination takes a general form in which a scale controls the shortcut and normalization is applied to the combined representation. Our reproduction examined fixed scales and normalization alternatives, which makes it the direct experimental link between that paper and LyASC.

Skip-scaling studies show that a coefficient that looks harmless algebraically can strongly change the signal as it passes through a deep network. Normalization can alter these outcomes, but it does not act as a stability controller. It rescales activations by a fixed rule, whereas LyASC uses a running local amplification indicator to set a branch mixture.

2.3 LEARNED GATES AND DYNAMIC ROUTING

Highway Networks introduced transform and carry gates, so learned control of information flow is already well established [6]. SkipNet and adaptive inference graph methods explore dynamic execution and routing, and stochastic depth randomly bypasses residual blocks during training. Because of this prior work, LyASC cannot be presented as the first adaptive skip mechanism.

What differs is the signal that drives the adaptation. Feature-based gates use content or learned activations. Dynamic routing methods usually aim to reduce compute. Stochastic depth follows a random schedule. LyASC instead uses a stability-related signal derived from local branch amplification. That is a difference in control criterion, and its value has to be shown empirically.

2.4 OPTIMIZATION STABILITY AND LYAPUNOV ANALYSIS

Research on the edge of stability studies how gradient-based optimization interacts with sharpness and stability thresholds [13]. It supports the broader premise that training dynamics can be measured. It does not prescribe per-block skip weights, and global curvature measures should not be mistaken for the local proxy used here.

Lyapunov exponents describe how infinitesimal perturbations grow or contract in a dynamical system. For neural training, finite-time indicators can be informative, but their meaning depends on the chosen state, operator, perturbation direction, normalization, and observation window. The implemented estimator is a finite, smoothed Jacobian-vector amplification proxy. It does not prove global stability, and it should not be called a complete classical Lyapunov exponent calculation unless the full trajectory-based definition is evaluated.

2.5 COMPARATIVE ANALYSIS

Table 2.1 summarizes the main literature families and how each relates to LyASC.

Table 2.1: Research families and their relationship to LyASC

Research family	Established contribution	Distinction from LyASC
Residual identity mappings	Direct information and gradient route	Fixed identity contribution in the standard block
Skip scaling and normalization	Demonstrates scaling sensitivity	Usually uses selected scaling rules rather than the project's online proxy
Learned gating	Learns branch contribution	Gate is not necessarily driven by a stability signal
Dynamic routing	Selects execution paths by input or policy	Often emphasizes compute efficiency
Edge-of-stability analysis	Characterizes optimizer dynamics	Usually does not control each residual block's gate
Lyapunov-style analysis	Quantifies perturbation growth in a dynamical system	LyASC tests a proxy as a controller input

2.6 EIGHTEEN-REFERENCE SURVEY MAP

The references are grouped by purpose. References 1 to 5 cover residual learning, identity mappings, scaling, normalization, and optimization. References 6 to 11 cover gating, dynamic routing, stochastic depth, and alternative skip controls. References 12 to 18 cover stability analysis, edge-of-stability behaviour, and Lyapunov-related training diagnostics. Some entries in the bibliography still need their publication details checked against publisher records before formal citation.

2.7 RESEARCH GAP

The literature shows that skip contributions affect signal propagation, that gating and routing can adapt information flow, and that training dynamics have measurable stability-related properties. The gap examined here sits where these directions meet: using a local, online amplification proxy to set a block-level identity/residual mixing coefficient. This is a scoped empirical claim. It does not say that no earlier work has used a stability-driven gating mechanism, and a formal novelty claim would need a comprehensive search of related work and a careful comparison.

CHAPTER 3

BASE PAPER REPRODUCTION

3.1 PURPOSE OF REPRODUCTION

We treated the base-paper reproduction as a separate validation phase before introducing LyASC. A new mechanism cannot be judged fairly if the reference model, training schedule, or evaluation code already differs from the published setup. The reproduced baseline checked that the training pipeline works and gave us measurements to compare LyASC against.

3.2 REFERENCE ARCHITECTURE AND DATASET

The experiments use CIFAR-10, a dataset of small colour images in ten classes. The baseline network is PreAct-ResNet-110. The reproduction covers several residual-scaling and normalization variants, and the reference model and its configuration are kept apart from the LyASC model so that debugging changes to the proposed model cannot quietly alter the baseline.

3.3 REPRODUCTION PROTOCOL

The baseline reproduction uses multiple random seeds and compares the resulting test errors with reference targets. Nine of the 10 Table 1 variants fell within the pre-registered tolerance bounds, with three seeds per variant. This shows that the pipeline reproduces most of the selected reference behaviour. It is not a claim of bitwise identity with the original implementation.

3.4 BASELINE RESULTS

The reproduced standard baselines differ noticeably. The 1xSkip baseline reached a mean test error of about 6.27%, 2rSkip+LN about 6.15%, and the learned wSkip+LN baseline about 8.16%. A learned gate therefore does not automatically beat a fixed scale, and normalization and branch scaling interact.

The stress configurations were especially informative. The fixed variants $\mathrm{LN}(0.5x + F(x))$,

$\mathrm{LN}(x+2F(x))$, and $\mathrm{LN}(x+3F(x))$ produced test errors of 37.14%, 37.59%, and 49.27%. These high errors motivate the stress-rescue experiments. They describe those particular configurations and say nothing general about residual networks.

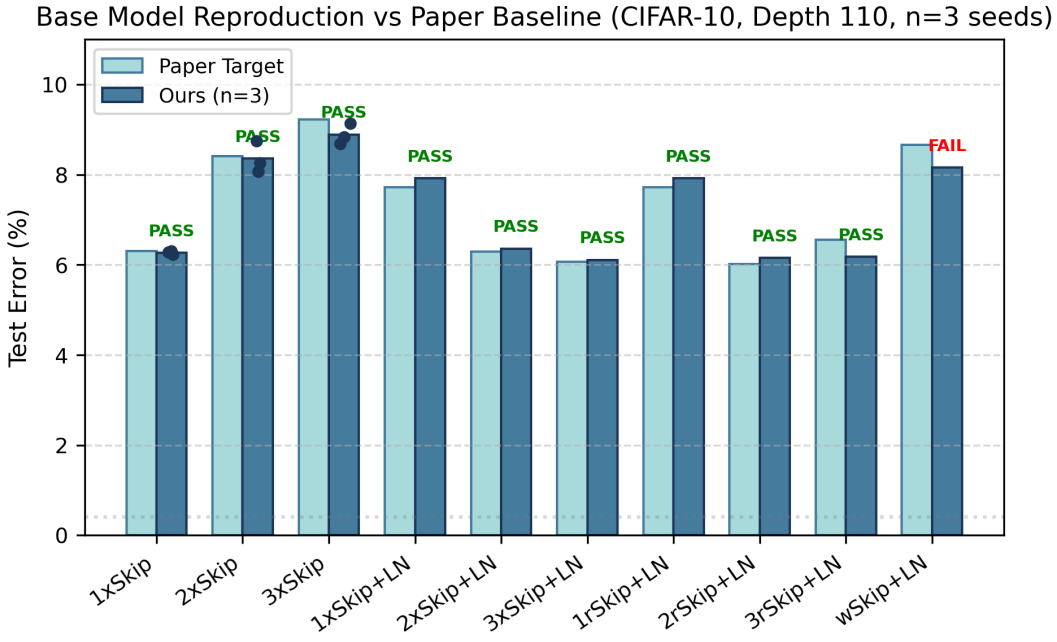


Figure 3.1: Reproduced base-paper results across selected Table 1 variants.

3.5 WHAT THE REPRODUCTION CONTRIBUTES TO LYASC

The baseline phase gives LyASC three things. It verifies the model and evaluation pipeline against known results, it identifies stress configurations where fixed branch scaling performs poorly, and it supplies a common protocol for testing the stability-informed gate. Without it, a low LyASC error could come from different training settings instead of the proposed mechanism.

3.6 REPRODUCTION LIMITATIONS

The reproduction covers a selected subset of the base paper, not every experiment across all architectures and datasets. Random-seed variability remains, and comparisons with studies that use different implementations should be made cautiously. The data support a controlled local comparison within this project. They do not settle how all skip-connection methods rank.

CHAPTER 4

LYASC METHOD AND MATHEMATICAL FORMULATION

4.1 OVERVIEW

LyASC adds a stability-informed gate to each residual block. A stability estimator computes a local amplification proxy from a probe vector and a Jacobian-vector product. The proxy is smoothed with an exponential moving average (EMA) and combined with a learnable per-block sensitivity parameter to produce a sigmoid gate. The gate mixes the identity and residual branches, and the chosen post-gate normalization is applied where configured.

4.2 RESIDUAL-BLOCK FORMULATION

A generalized LyASC block can be written as

$$y_l = G\left(\alpha_l s_x x_l + (1-\alpha_l) s_F F_l(x_l;\theta_l)\right),$$

where s_x and s_F are configured scale factors, G is the post-gate operation, and $\alpha_l \in (0,1)$ is the gate. In the standard formulation used to explain the concept, $s_x = s_F = 1$. The stress tests change the branch scaling to reproduce failure modes from the reference study.

The identity contribution is $\alpha_l s_x x_l$, and the residual contribution is $(1-\alpha_l) s_F F_l(x_l;\theta_l)$. When α_l is near one, the identity branch gets more weight. When it is near zero, the residual branch does. The gate acts as a convex mixing coefficient only when the scale factors are nonnegative and the branches are read before the optional normalization step.

4.3 STABILITY PROXY

Let J_l denote the local Jacobian of the residual branch at block l. A probe direction v_l is passed through a Jacobian-vector product, and the norm of the result estimates local amplification. The raw proxy is based on a logarithmic norm term. The implementation uses a finite number of estimator operations and an EMA to damp high-frequency variation. The exact estimate depends

on the probe, the mini-batch, the current parameters, and the update step.

A classical finite-time Lyapunov exponent follows perturbation growth across a sequence of operators and accumulates normalized log-growth factors. The estimator here is described more narrowly as a local amplification proxy, because a single JVP norm does not establish the full asymptotic or trajectory-based exponent. The distinction matters for scientific accuracy.

4.4 EXPONENTIAL SMOOTHING

The smoothed proxy is

$$\bar{q}_{l,t} = \rho \bar{q}_{l,t-1} + (1 - \rho) q_{l,t},$$

where $q_{l,t}$ is the raw estimate and ρ is the EMA decay. The project configuration uses $\rho = 0.9$, estimator setting $k = 1$, and a window/iteration setting $N = 20$, as recorded in the repository materials. Smoothing reduces noise from mini-batch variation. It also adds lag and leaves the estimator's bias in place.

4.5 ADAPTIVE GATE

The gate is

$$\alpha_l = \sigma(z_l), \qquad \sigma(z) = \frac{1}{1 + \exp(-z)},$$

where z_l combines the smoothed stability proxy with a learnable per-block sensitivity parameter and the configured reference threshold. In conceptual form,

$$\alpha_l = \sigma\left(\beta_l(\bar{q}_l - q_0)\right).$$

The sign and parameterization of β_l set the direction of the gate response. In the reported implementation the sensitivity is not constrained to be positive, so a larger proxy does not necessarily raise the identity weight. The model learns the response within the implemented parameterization.

The architecture notes initialize the sensitivity parameter to zero, which makes the initial sigmoid gate equal to 0.5. Both branches then start with equal nominal weight before the sensitivity evolves. The model need not stay at equal weighting as training proceeds.

4.6 CONTROLLER INTERPRETATION

LyASC is feedback-inspired. It measures a local signal, maps it to a gate, combines the branches, and repeats as the model parameters change. It carries no formal control-theoretic stability guarantee. Nothing shows that the estimator observes all unstable modes or that the learned gate always moves in a stabilizing direction. The project tests whether the mechanism is useful in practice.

4.7 INTEGRATION WITH PREACT-RESNET-110

The gate is inserted into the residual blocks and the overall PreAct-ResNet-110 structure is preserved. Baseline model code stays separate, and model variants are exposed through configuration. That supports comparisons across standard skip, fixed scaling, learned gating, and LyASC. Post-gate normalization is treated as a controlled factor, since the base paper suggests that normalization can change the outcome of scaled residual additions.

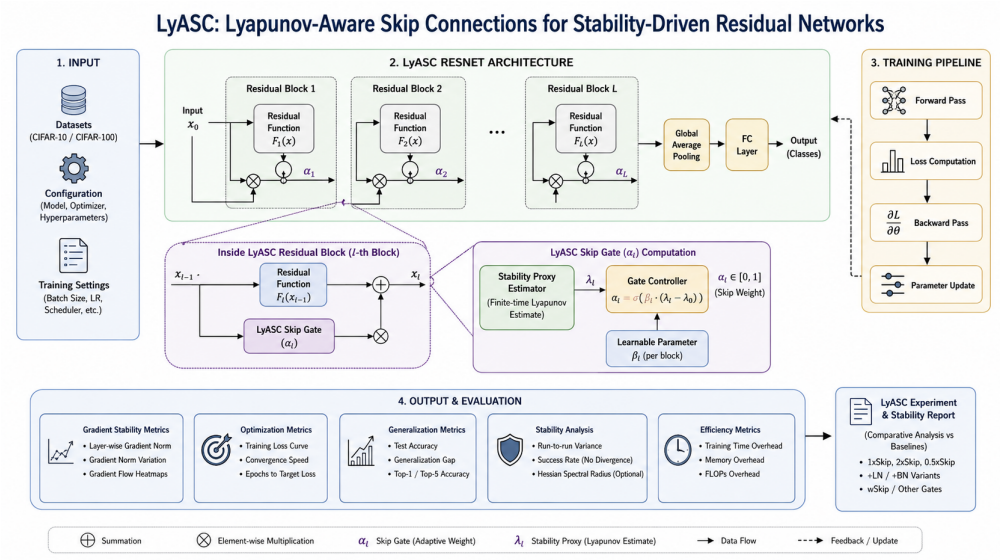


Figure 4.1: LyASC block architecture showing identity, residual, and stability-estimation paths.

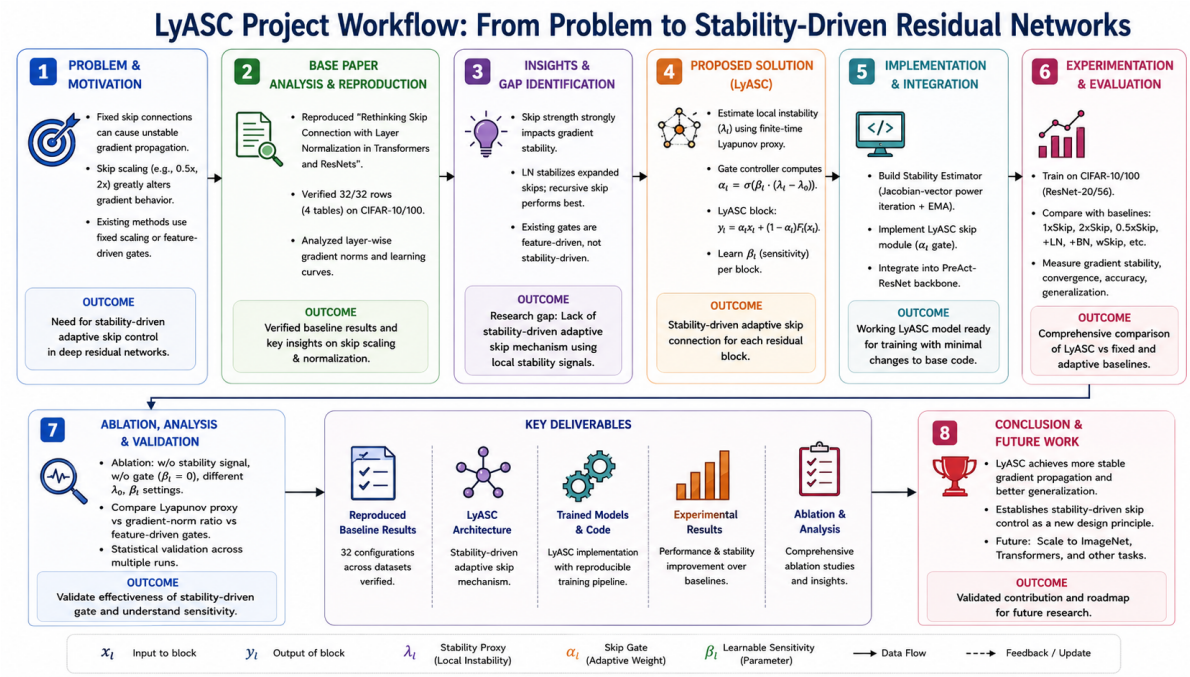


Figure 4.2: Project workflow from baseline reproduction through stress testing and ablation.

4.8 METHOD SUMMARY

The method has four steps: evaluate the residual block, estimate local branch amplification, smooth the estimate and turn it into a gate, and combine the identity and residual contributions. The experiments compare this procedure with the reproduced fixed baselines, including stress configurations where those baselines show high error.

CHAPTER 5

SYSTEM DESIGN AND IMPLEMENTATION

5.1 SYSTEM ARCHITECTURE

The system has a baseline model, a LyASC model, a training pipeline, an experiment configuration layer, a result logger, and dashboard and reporting artifacts. The baseline implementation stays separate so it can serve as a stable reference. LyASC-specific logic lives in the model and stability-estimator modules, and the training scripts select the required configurations and record outcomes.

5.2 SOFTWARE ENVIRONMENT

The repository materials list Python 3.10 and PyTorch 2.x as the main environment. Experiments ran in an isolated Conda environment named `lyasc-experiments`. CUDA availability and hardware configuration were checked before training, and an environment guard was added so experiments would not run under an unintended system interpreter.

5.3 STABILITY ESTIMATOR MODULE

The estimator produces a block-local proxy and keeps its smoothed state. Its buffers must live on the same device as the model parameters. An earlier mismatch between CPU buffers and CUDA tensors was fixed by binding the estimator state explicitly to the target device. The project materials also record import-precedence safeguards that make experiment scripts import the intended model implementation.

5.4 GATE AND MODEL INTEGRATION

The gate combines the smoothed proxy with a per-block sensitivity parameter, weights the identity and residual branches, and optionally applies post-gate normalization. Because of this modular design, the same backbone can be tested with no post-gate operation, LayerNorm2d, or

BatchNorm2d. Branch scales can also be changed in stress configurations without touching the estimator's basic interface.

5.5 TRAINING AND EVALUATION PIPELINE

The pipeline handles training, validation, test evaluation, checkpointing, logging, and resumption after interrupted sessions. Results are stored as structured JSON and summarized into tables and figures. MLflow records experiment parameters, metrics, and run metadata in a SQLite backend, which allows runs to be compared and preserves the provenance of each reported number.

5.6 TESTING AND VERIFICATION

The repository reports 78 passing tests across the combined base-model and LyASC suites. They cover estimator updates, skip-connection logic, and model integration. Passing unit tests show software-level consistency for the tested behaviours. They do not validate the scientific hypothesis, and they do not guarantee that every result reproduces on other hardware.

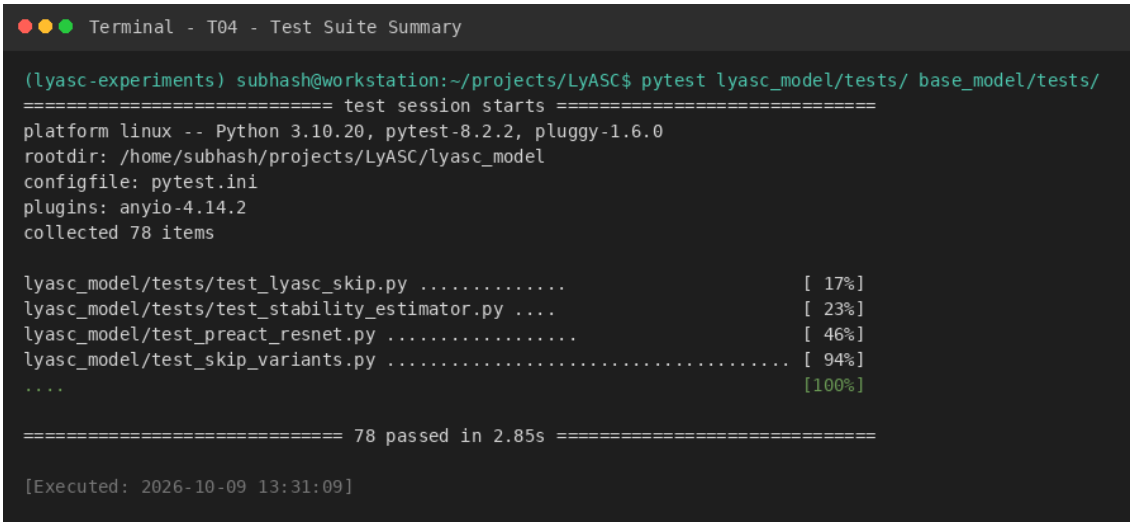


Figure 5.1: Terminal evidence for the automated test suite.

5.7 EXPERIMENT TRACKING AND DASHBOARD

MLflow retains training metrics and run parameters, and a static dashboard presents the main comparisons. The dashboard is a reporting layer and adds no new experimental evidence, so reported metrics should be traced back to the run logs or result tables. The project materials include screenshots of the experiment list, the run table, and the stress-test comparison.

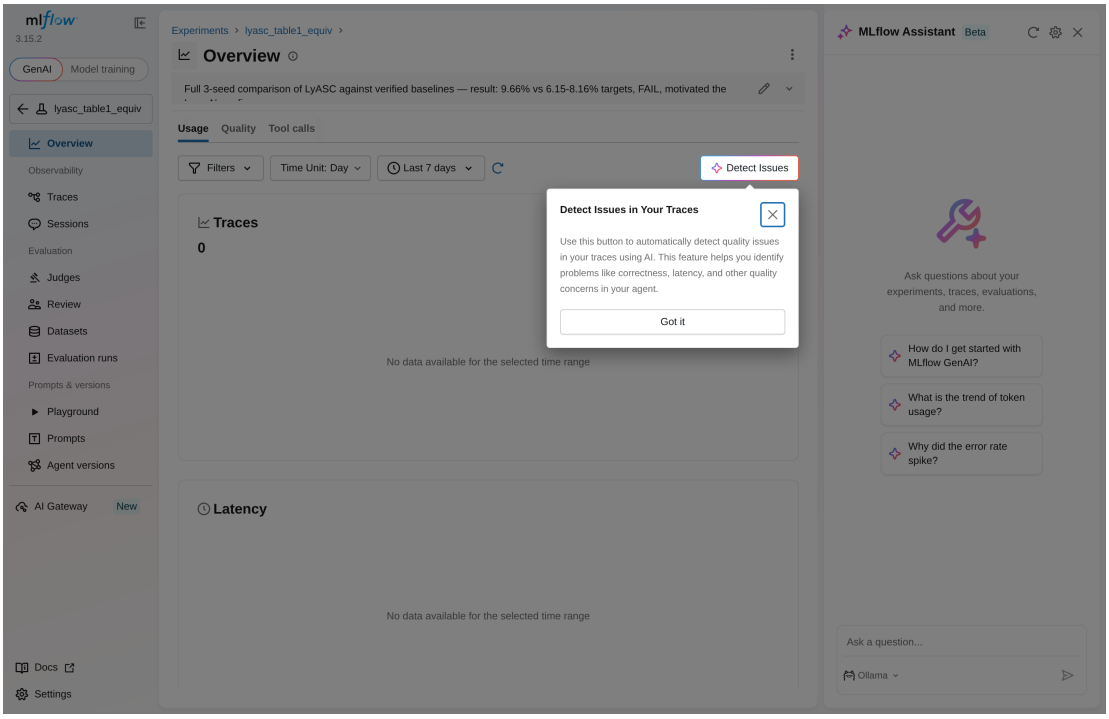


Figure 5.2: MLflow experiment runs table used to track experimental executions.

5.8 REPRODUCIBILITY SAFEGUARDS

Several problems came up during implementation and were fixed: import precedence, a configuration reverted in the baseline folder, a device mismatch in estimator buffers, inconsistent MLflow database paths, and timing discrepancies from interrupted and resumed runs. The materials document each issue and its fix. Without this record, a result table could mix metrics from unintended code paths or inconsistent run states.

CHAPTER 6

EXPERIMENTAL DESIGN AND RESULTS

6.1 EXPERIMENTAL QUESTIONS

The experiments address five questions:

1. Does the baseline implementation reproduce the main reference trends?
2. How does LyASC perform under ordinary CIFAR-10 training?
3. Can LyASC reduce error in stress configurations where fixed branch scaling fails?
4. How do initialization and post-gate normalization affect the gate and the model?
5. What are the runtime costs and seed-level limitations?

6.2 EXPERIMENTAL PROTOCOL

The main model is PreAct-ResNet-110 trained on CIFAR-10. Results are reported as test error, with mean and standard deviation where several seeds exist. Most configurations use three seeds, some ablations use two, and stress_3.0 uses five. Since seed counts differ, comparisons should be read as descriptive and not as statistical rankings.

6.3 BASELINE REPRODUCTION RESULTS

In the reproduced Table 1 results, 9 of 10 variants fell within the specified tolerance. The baseline errors that matter for the later stress tests are 37.14% for $\mathrm{LN}(0.5x + F(x))$, 37.59% for $\mathrm{LN}(x + 2F(x))$, and 49.27% for $\mathrm{LN}(x + 3F(x))$. Under standard conditions, 1xSkip reached 6.27% $\pm$ 0.04%, 2rSkip+LN reached 6.15%, and wSkip+LN reached 8.16%.

6.4 PHASE C: UN-NORMALIZED LYASC

The first LyASC variant, with no post-gate normalization, reached a mean test error of 9.66% $\pm$ 0.73% across three seeds. A later LayerNorm ablation lowered the error in a single seed, which led to the more systematic cold-start and normalization phases. The first result showed that a stability-informed gate alone could not match the stronger fixed baselines.

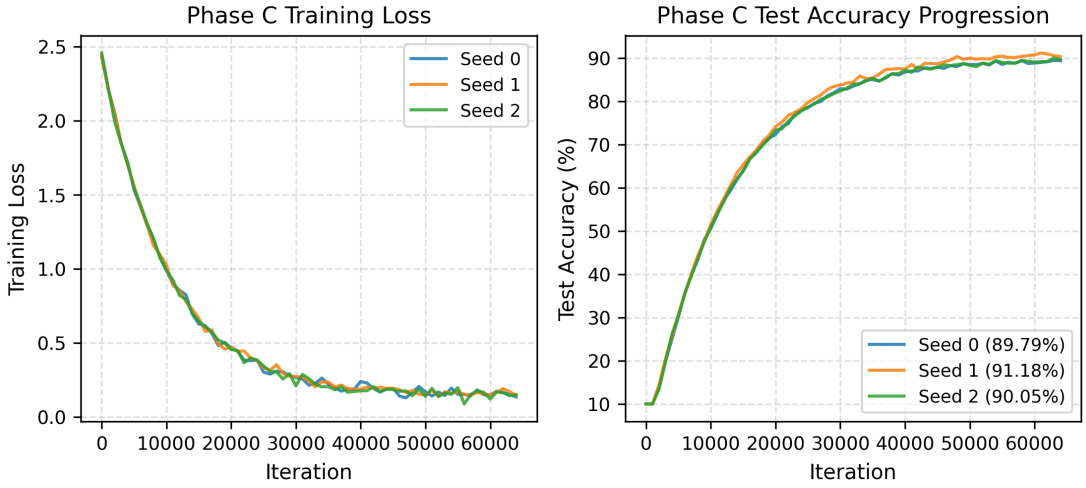


Figure 6.1: Phase C training curves for the un-normalized LyASC configuration.

6.5 PHASE D: COLD-START STRATEGY ABLATION

Phase D compared three initialization strategies with LayerNorm2d after the gate. The repository summary reports the following results:

Table 6.1: Phase D cold-start strategies

Strategy	Norm	Mean test error	Best seed
None ($w = 0$)	LayerNorm2d	$6.34\% \pm 0.15\%$	6.16%
Damped	LayerNorm2d	$6.37\% \pm 0.03\%$	6.35%
Warmup	LayerNorm2d	$6.40\% \pm 0.08\%$	6.31%

We chose the no-delay strategy as the practical default. Its mean was similar to or slightly lower than the others, and it needs no separate warmup procedure. The differences are small, the means are close, and each rests on three seeds, so they should not be over-interpreted.

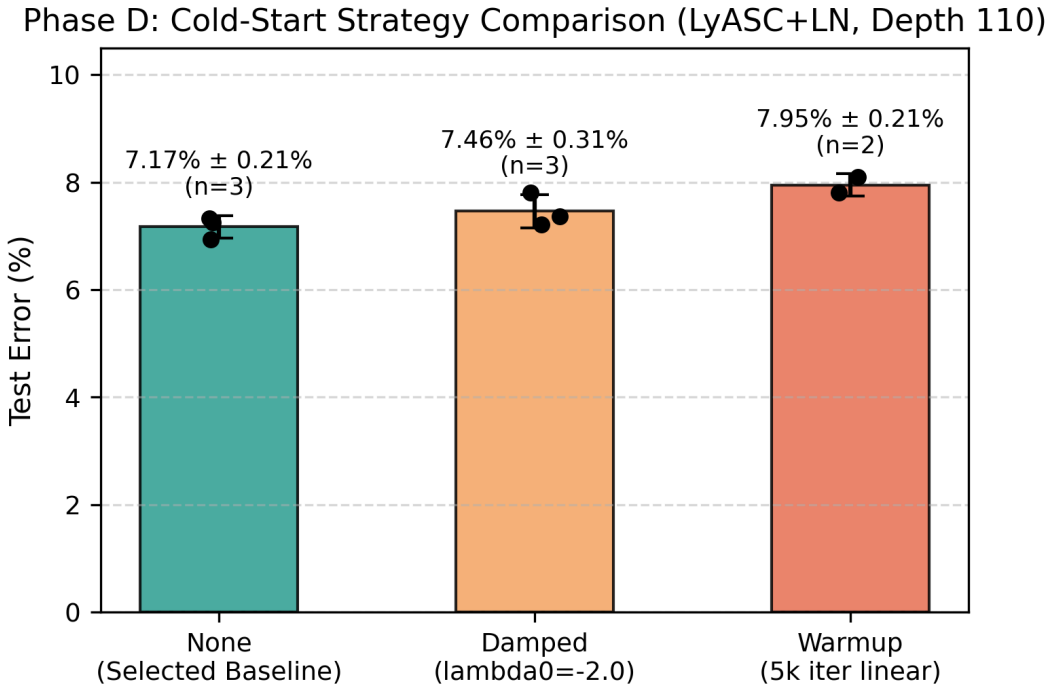


Figure 6.2: Cold-start strategy comparison.

6.6 PHASE E1: STRESS TESTING

The stress tests apply the branch-scaling conditions that produced high errors in the reproduced baseline. LyASC+LN with the selected cold-start strategy reached $8.38\% \pm 0.35\%$ under stress_0.5 and $8.44\% \pm 0.28\%$ under stress_2.0. Each of the three runs in these settings finished below 9% test error.

Table 6.2: Stress-test comparison

Stress variant	Fixed baseline error	LyASC mean error	Observed reduction
Stress_0.5	37.14%	$8.38\% \pm 0.35\%$	28.76 percentage points
Stress_2.0	37.59%	$8.44\% \pm 0.28\%$	29.15 percentage points
Stress_3.0	49.27%	mixed outcomes	see text

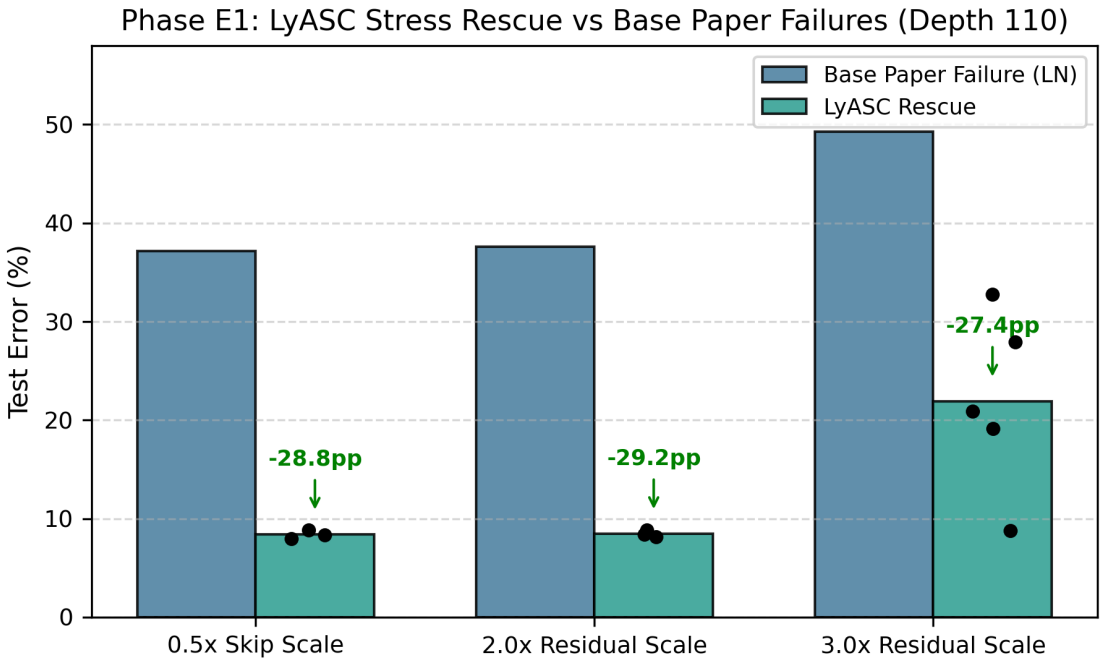


Figure 6.3: Stress-test error comparison between fixed baselines and LyASC.

6.7 STRESS_3.0: OUTCOME CLUSTERS

Stress_3.0 varied much more. One of the five seeds reached 8.75% test error. The other four had errors between 19.12% and 32.73%, with a mean of 25.16% and a median of 20.88%. The source materials give a pooled five-seed mean of 21.88% $\pm$ 9.15 percentage points, but a single pooled mean hides the two clusters. We therefore list the individual outcomes and separate full rescue from partial rescue.

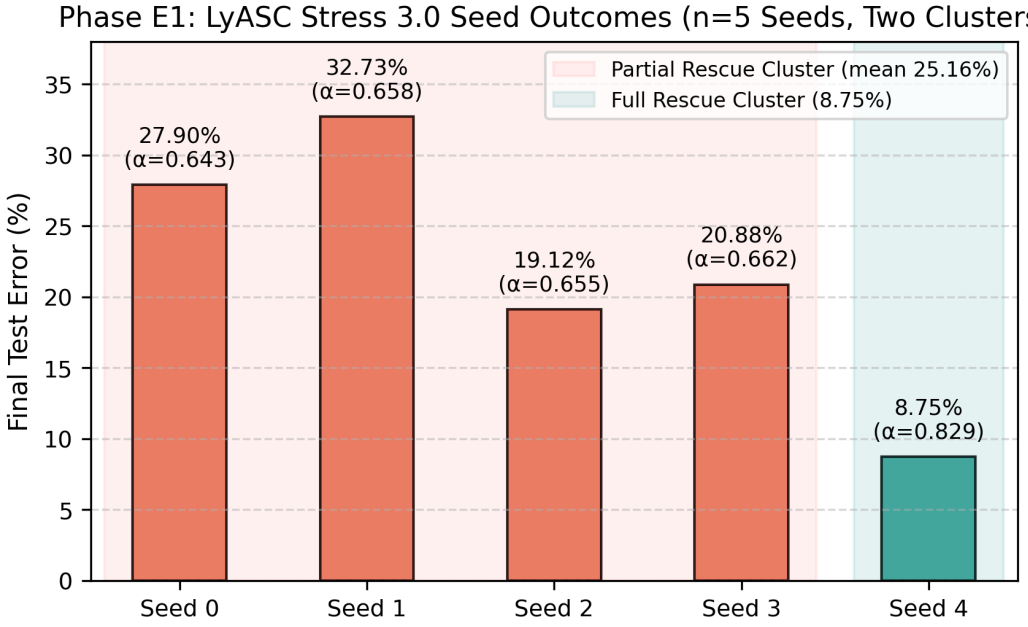


Figure 6.4: Stress_3.0 outcomes grouped into full- and partial-rescue clusters.

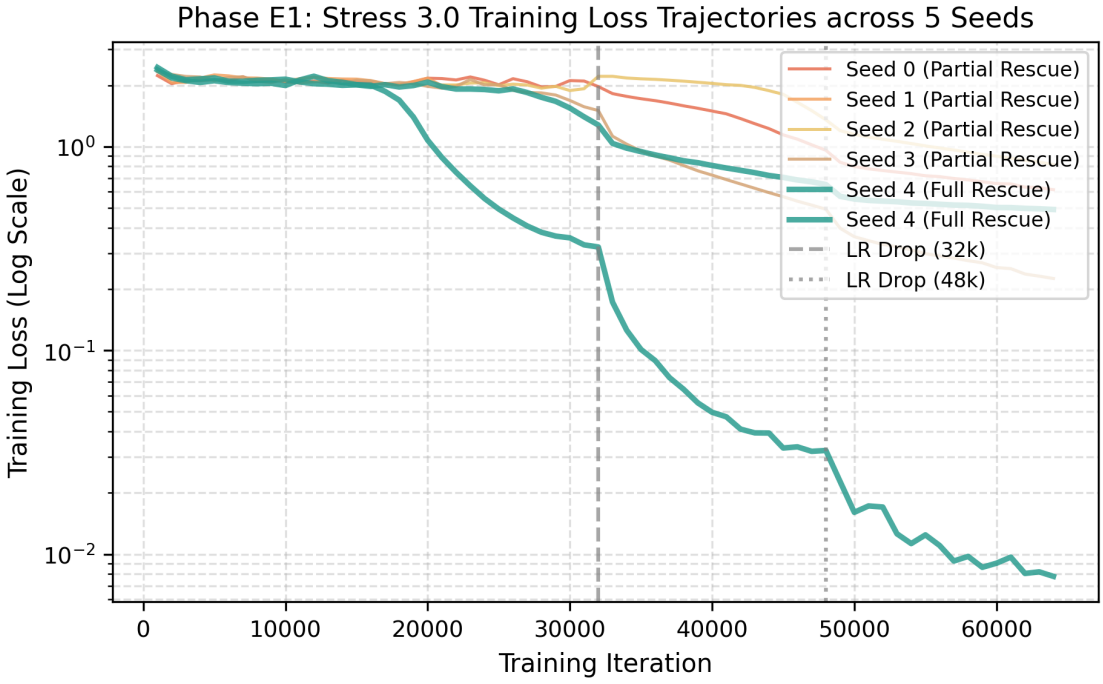


Figure 6.5: Training-loss trajectories for stress_3.0 runs.

The gate trajectories suggest that early optimization progress may be linked to the final out-

come. In the full-rescue run, the mean gate passed about 0.706 around iteration 16,000 and later reached 0.8285. The four partial-rescue runs plateaued around 0.64 to 0.66. Within that group, seeds that escaped the plateau before the learning-rate drop near iteration 32,000 ended with lower error than seeds that stalled or relapsed around the drop.

This pattern suggests a follow-up hypothesis about early gate progression, but it does not establish causality. The gate may influence optimization, track it, or both. More seeds and controlled interventions are needed before the apparent threshold can be treated as a reliable decision rule.

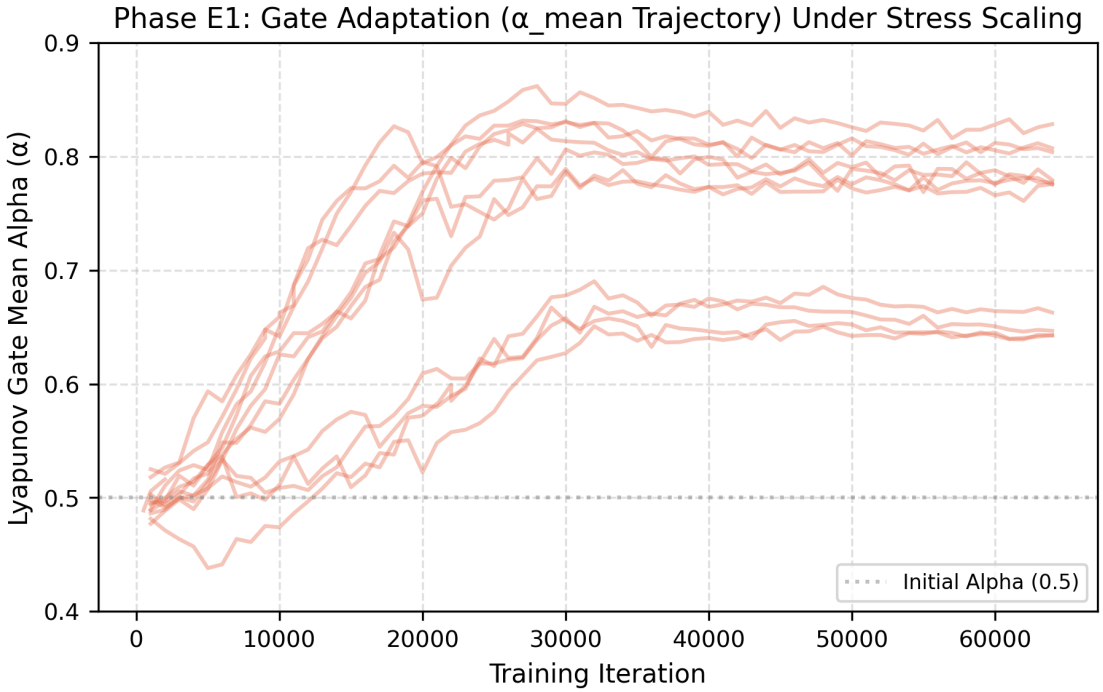


Figure 6.6: Mean gate trajectories under different stress levels.

6.8 PHASE E2: POST-GATE NORMALIZATION

Phase E2 compared identity (no normalization), LayerNorm2d, and BatchNorm2d after the gate. The no-normalization configuration reached 9.52% $\pm$ 0.98% across two seeds, against 7.17% $\pm$ 0.21% for the closed-out LyASC+LN baseline across three seeds, a difference of 2.35 percentage points. LyASC+BN reached 6.34% $\pm$ 0.01% across two seeds in unstressed training. That is 0.83

percentage points lower than LyASC+LN, but the seed counts differ and the BatchNorm variant has not been tested under the stress conditions.

Table 6.3: Post-gate normalization pairing

Variant	Post-gate operation	Seeds	Mean error
LyASC+LN	LayerNorm2d	3	$7.17\% \pm 0.21\%$
LyASC_noG	Identity	2	$9.52\% \pm 0.98\%$
LyASC+BN	BatchNorm2d	2	$6.34\% \pm 0.01\%$
PreAct-ResNet-110	Not applicable	3	$6.27\% \pm 0.04\%$

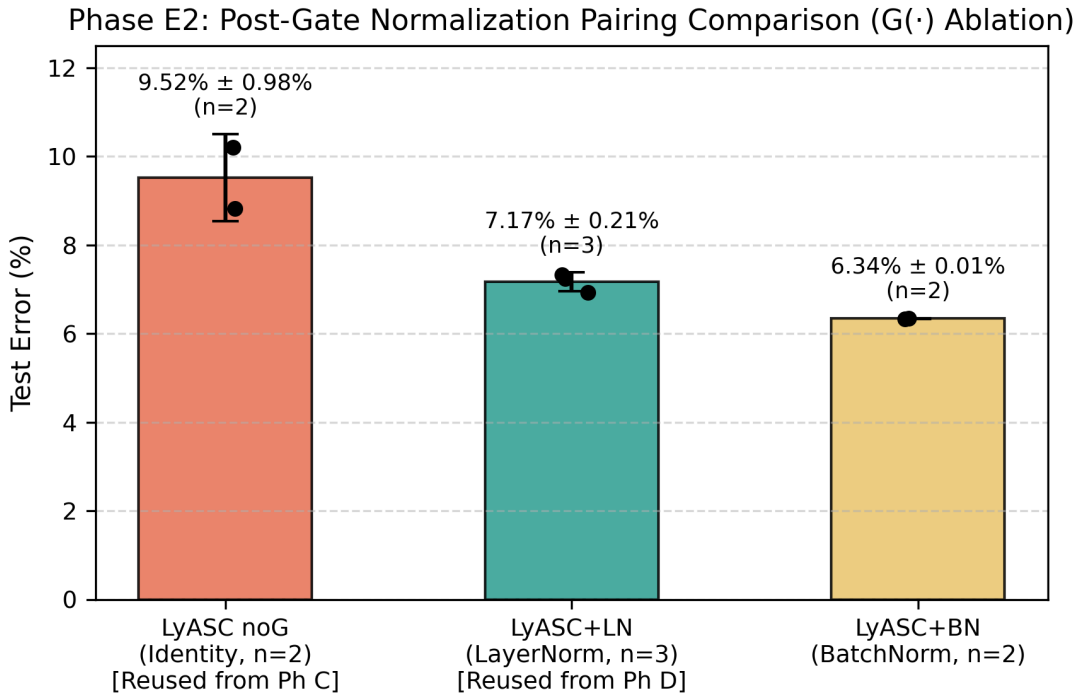


Figure 6.7: Comparison of post-gate normalization variants.

6.9 RUNTIME AND TRACKING

The source materials report per-iteration runtime rising from about 0.261 seconds for un-normalized LyASC to about 0.498 seconds in one timing summary for LyASC+LN. A resumed-run audit reported about 0.684 seconds per iteration for a long run. The two figures come from different timing contexts and should not be merged into one number. The project materials attribute much

of the overhead to repeated LayerNorm2d operations across residual blocks, and the cost of the estimator alone was not conclusively measured.

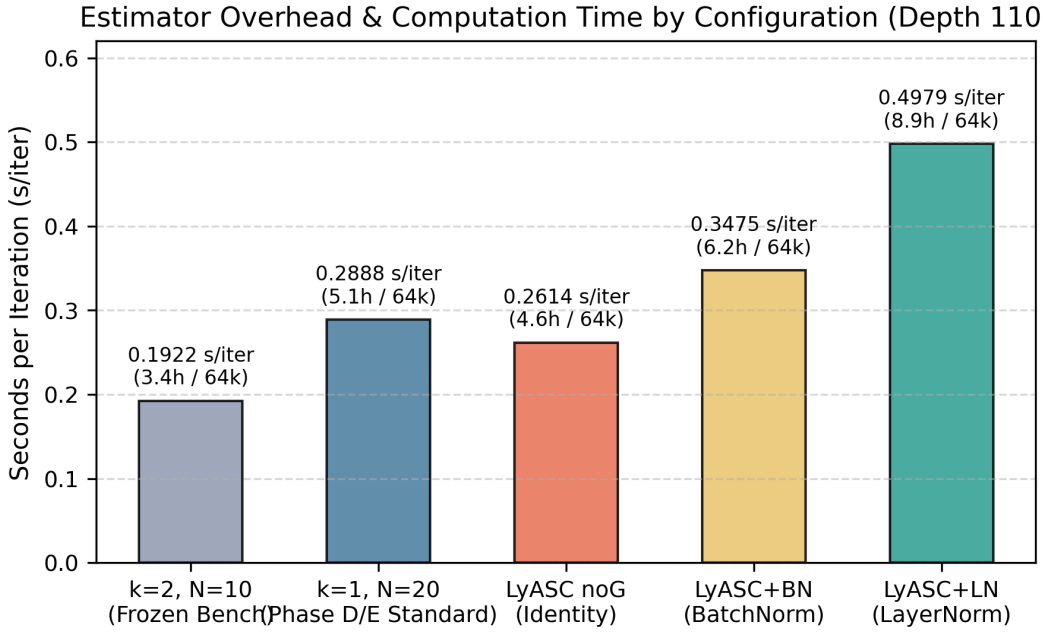


Figure 6.8: Timing and overhead comparison across model configurations.

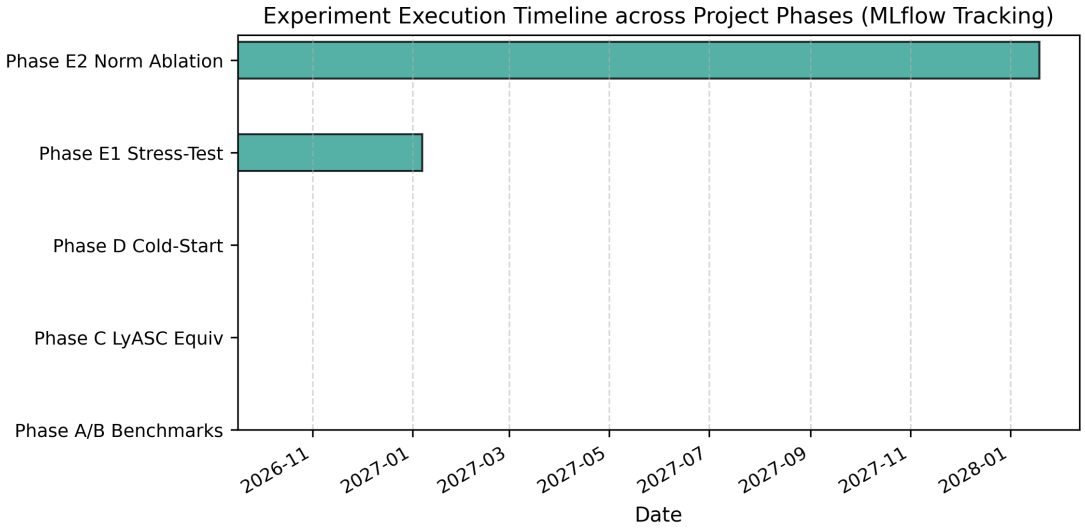


Figure 6.9: Experiment timeline recorded across project phases.

6.10 RESULTS SUMMARY

The main findings are:

- The base-paper pipeline reproduced 9 of 10 selected Table 1 variants within tolerance.
- Under stress_0.5 and stress_2.0, every reported LyASC seed had a test error below 9%.
- Stress_3.0 stayed difficult, with one full-rescue seed and four partial-rescue seeds.
- Post-gate normalization changed the observed results, and the no-normalization variant did worse than the LayerNorm baseline.
- BatchNorm gave a promising unstressed mean but has not been evaluated under the stress settings.
- LyASC+LN did not beat the best fixed baseline in normal-condition training.

CHAPTER 7

DISCUSSION, LIMITATIONS, AND VALIDITY

7.1 INTERPRETATION OF MAIN FINDINGS

The results show that a stability-informed gate is feasible in a residual network. The strongest evidence is the stress_0.5 and stress_2.0 comparison, where the fixed baselines had high error and the LyASC runs converged to single-digit error. That is a meaningful result for the tested configuration. It does not show that LyASC is universally better, and it does not make the local proxy a complete measure of network stability.

7.2 NORMAL-CONDITION PERFORMANCE

Under standard training, LyASC+LN reached 7.17% mean error. The best fixed baseline in the reported comparisons, 2rSkip+LN, reached 6.15%, a gap of about 1.02 percentage points. LyASC improves on the reproduced wSkip+LN learned-gate baseline at 8.16% but does not beat every fixed schedule. A practical controller has to be judged on ordinary-condition accuracy and computational cost as well as on recovery from stress.

7.3 INTERPRETATION OF STRESS_3.0

The stress_3.0 outcomes are heterogeneous, and the pooled mean alone would hide the large spread. The apparent link between early gate movement and final performance is useful for forming a hypothesis, but it needs more controlled experiments. One follow-up is to compare a controller that changes gate learning dynamics explicitly against the current implementation, holding seeds and training schedules constant.

7.4 ESTIMATOR VALIDITY

The estimator is a finite, smoothed local amplification proxy based on a Jacobian-vector product. It depends on the probe direction, the selected layer, the mini-batch, the parameter state, and

the update interval. A positive proxy value does not prove that a block is unstable in a formal dynamical-systems sense, and a negative value does not prove global contraction. Future work should compare the proxy with multiple perturbation directions, finite-difference trajectories, Jacobian norm estimates, and independent stability diagnostics.

7.5 STATISTICAL LIMITATIONS

Several configurations use only two or three seeds, and stress_3.0 uses five. These sample sizes are adequate for an initial engineering study but limit what can be said about population-level variability. Future work should fix seed counts in advance, report confidence intervals, keep individual-seed results, and run significance or equivalence tests where appropriate. Comparisons across different seed counts and experiment phases are observed differences and do not establish superiority.

7.6 COMPUTATIONAL COST

The normalized LyASC model takes longer per iteration. The timing measurements come from different execution contexts, including a resumed run, so they cannot be swapped for one another. A clean benchmark should separate estimator cost from normalization cost, synchronize GPU operations when timing CUDA execution, warm up the model, and report the median and dispersion over repeated timing windows.

7.7 REPRODUCIBILITY AND INTERNAL VALIDITY

The project documented several implementation issues and their fixes: Python import precedence, an altered baseline configuration that was reverted, a CPU/GPU estimator-state mismatch, inconsistent MLflow database paths, and a timing misreading caused by a resumed session. Documenting them adds transparency, and it also means a future reproduction should use the final checked configuration and the raw result files instead of relying on a summary slide or dashboard.

7.8 EXTERNAL VALIDITY

The results are limited to CIFAR-10 and PreAct-ResNet-110. They cannot be assumed to carry over to CIFAR-100, ImageNet, other depths, other optimizers, or Transformer architectures. The estimator's computational profile may also change with width, batch size, hardware, and implementation. These remain open evaluation questions.

7.9 THREATS TO VALIDITY

Possible threats include seed variability, hyperparameter selection, differences between the reproduced baseline and the original authors' environment, measurement noise in the estimator, interaction between normalization and gating, and incomplete stress testing of the BatchNorm pairing. Baseline separation, run tracking, and multiple seeds reduce some of these threats but do not remove them.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

This report presented LyASC, a stability-informed skip-mixing method for PreAct-ResNet-110. We first reproduced skip-scaling and normalization experiments from the base paper, then implemented a per-block gate driven by a smoothed local Jacobian-vector amplification proxy. The evaluation covered normal training, severe branch-scaling stress tests, cold-start strategies, post-gate normalization, and runtime behaviour.

LyASC handled the tested stress_0.5 and stress_2.0 configurations far better than the corresponding fixed baselines, with mean test errors of 8.38% and 8.44% against 37.14% and 37.59%. Stress_3.0 gave a more complicated picture: one seed reached 8.75%, and the other four achieved only partial rescue. In normal conditions, LyASC+LN reached 7.17%, behind the best fixed baseline at 6.15%. The evidence supports a mechanism that helps under some conditions. It does not support a universal performance claim.

8.2 KEY CONTRIBUTIONS

The project contributes:

1. A reproduced baseline suite for selected skip-scaling and normalization configurations.
2. A stability estimator based on local Jacobian-vector amplification with temporal smoothing.
3. A block-level sigmoid gate that mixes identity and residual contributions.
4. Stress tests that expose both successful rescue and seed-sensitive partial rescue.
5. Cold-start and post-gate normalization ablations.

6. A tracked implementation with structured metrics, checkpoints, unit tests, and visual reporting artifacts.

8.3 FUTURE WORK

Future work should first add seeds for stress_3.0 and check whether the early gate-progression pattern holds. Estimator overhead should then be benchmarked separately from normalization, and LyASC+BN should be tested under stress conditions. The estimator window, smoothing coefficient, threshold, sensitivity parameterization, and gate learning rate should be varied systematically. The method should be evaluated on CIFAR-100 and other network depths before any claim of generality. Comparisons with feature-driven routing, fixed scaling, and constrained-gate alternatives should use matched training budgets and common evaluation protocols.

BIBLIOGRAPHY

[1] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.

[2] K. He, X. Zhang, S. Ren, and J. Sun, “Identity Mappings in Deep Residual Networks,” in *Proc. European Conference on Computer Vision*, 2016, pp. 630–645, doi: 10.1007/978-3-319-46493-0_38.

[3] F. Liu, X. Ren, Z. Zhang, X. Sun, and Y. Zou, “Rethinking Skip Connections with Layer Normalization in Transformers and ResNets,” in *Proc. COLING*, 2020, pp. 3996–4007, arXiv:2105.07205.

[4] A. Zaeemzadeh, N. Rahnavard, and M. Shah, “Norm-Preservation: Why Residual Networks Can Become Extremely Deep?” bibliographic details and DOI should be verified against the publisher record before submission.

[5] L. E. MacDonald, J. Valmadre, H. Saratchandran, and S. Lucey, “On Skip Connections and Normalisation Layers in Deep Optimisation,” conference/workshop record, 2023; verify final venue and DOI before submission.

[6] R. K. Srivastava, K. Greff, and J. Schmidhuber, “Highway Networks,” 2015, arXiv:1505.00387.

[7] X. Wang, F. Yu, Z. Dou, T. Darrell, and J. E. Gonzalez, “SkipNet: Learning Dynamic Routing in Convolutional Networks,” in *Proc. ECCV*, 2018, doi: 10.1007/978-3-030-01261-8_25.

[8] A. Veit and S. Belongie, “Convolutional Networks with Adaptive Inference Graphs,” *International Journal of Computer Vision*, 2020; verify final DOI and metadata.

[9] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, “Deep Networks with Stochastic Depth,” in *Proc. ECCV*, 2016, doi: 10.1007/978-3-319-46493-0_39.

[10] F. A. Joseph, J. Sieber, M. N. Zeilinger, and C. Amo Alonso, "Lambda-Skip Connections: The Architectural Component That Prevents Rank Collapse," arXiv:2410.10609, 2024; verify final publication metadata.

[11] Y. R. Thota, "CosineGate: Semantic Dynamic Routing via Cosine Incompatibility in Residual Networks," arXiv:2512.22206; verify title, author, date, and public record before submission.

[12] K. Nar and S. Sastry, "Residual Networks: Lyapunov Stability and Convex Decomposition," arXiv:1803.08203, 2018.

[13] J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar, "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability," in *Proc. ICLR*, 2021, arXiv:2103.00065.

[14] S. Arora, Z. Li, and A. Panigrahi, "Understanding Gradient Descent on the Edge of Stability in Deep Learning," arXiv:2205.09745, 2022; verify venue and author metadata.

[15] A. Damian, E. Nichani, and J. D. Lee, "Self-Stabilization: The Implicit Bias of Gradient Descent at the Edge of Stability," arXiv:2209.15594, 2022/2023; verify final venue and DOI.

[16] A. Andreyev and P. Beneventano, "Edge of Stochastic Stability: Revisiting the Edge of Stability for SGD," arXiv:2412.20553; verify final metadata.

[17] T. Mittra, "Utilizing Lyapunov Exponents in Designing Deep Neural Networks," arXiv:2410.05988, 2024; verify metadata and methodological relevance.

[18] N. Sharafi, C. Martin, and S. Hallerberg, "On the Weight Dynamics of Learning Networks," arXiv:2405.00743, 2024; verify title, authors, and publication status.

CHAPTER A

APPENDIX - SUPPLEMENTARY MATERIAL

A.1 KEY CONFIGURATION VALUES

The main estimator settings are $k = 1$, $N = 20$, and EMA decay 0.9. The main experiments use PreAct-ResNet-110 on CIFAR-10. The three stress conditions are the contracted skip condition $\mathrm{LN}(0.5x + F(x))$, the expanded residual condition $\mathrm{LN}(x + 2F(x))$, and the stronger expanded residual condition $\mathrm{LN}(x + 3F(x))$.

A.2 PER-SEED STRESS RESULTS

Table A.1: Individual-seed stress results

Condition	Seed	Test error
Stress_0.5	0	7.98%
Stress_0.5	1	8.84%
Stress_0.5	2	8.31%
Stress_2.0	0	8.13%
Stress_2.0	1	8.81%
Stress_2.0	2	8.38%
Stress_3.0	0	27.90%
Stress_3.0	1	32.73%
Stress_3.0	2	19.12%
Stress_3.0	3	20.88%
Stress_3.0	4	8.75%

A.3 IMPLEMENTATION VERIFICATION

The project materials report 78 passing tests in 5.74 seconds, covering the baseline and LyASC components. This is a software verification result and is kept apart from the empirical model-performance claims.

A.4 EVIDENCE ARTIFACT SELECTION

The report includes the architecture and workflow diagrams, the baseline reproduction plot, the cold-start comparison, the stress rescue comparison, the stress_3.0 outcome clusters and loss curves, the gate trajectories, the normalization pairing, the runtime chart, the experiment timeline, and selected test and tracking screenshots. Each supports the methodology, a major result, the implementation verification, or a limitation. Duplicate screenshots and raw terminal captures that added no distinct evidence were left out of the main chapters.

A.5 REFERENCE VERIFICATION NOTE

The literature shortlist contains 18 intended references. Before journal submission, check the exact title, author order, venue, year, DOI, and version of each entry against the publisher or official preprint record. This matters most for recent or less-established titles and for preprints whose metadata may have changed. The reference list preserves the intended 18-topic coverage and is not a substitute for that final check.